

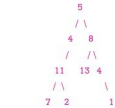
the information obtained from these to find the SCCs. These hints should be more than sufficient for readers to solve the problem. So do send in your solutions before the end of this month.

This month's programming question

This month we will revisit binary trees. A binary tree is made of tree nodes, where each node contains a 'left' pointer, a 'right' pointer, and a data element. The 'root' pointer points to the topmost node in the tree. The left and right pointers themselves recursively point to smaller 'sub-trees' on either side. A null pointer represents a binary tree with no elements, namely, the empty tree.

You are given a binary tree containing a set of integer values, with no duplicates. There exist paths in the binary tree from the root to the leaves. Each path is distinguished by the set of nodes contained in the path. A root-to-leaf-path consists of a sequence of nodes in a tree, starting with the root node and proceeding downward to a leaf (a node with no children). An empty tree contains no root-to-leaf paths.

So, for example, the following tree has exactly four root-to-leaf paths:



Root-to-leaf paths:

path 1: 5 4 11 7
 path 2: 5 4 11 2
 path 3: 5 8 13
 path 4: 5 8 4 1

We will consider the question of finding the sum of the values along each root-to-leaf path. For example, the sum of the values on the 5-4-11-7 path is $5 + 4 + 11 + 7 = 27$. We call the sum of values along the root-to-leaf path as the path-sum value. For instance, the path-sum values along the four paths in the binary tree above are 27, 22, 26 and 18.

Let us consider the problem associated with this, namely, given a sum value X and a binary tree, does there exist a root-to-leaf-path in the binary tree such that the path-sum value is equal to X ? For instance, if the sum value is 22, we can say that there exists no root-to-leaf-path in the above binary tree, which has a path-sum value equal to this. Our intention is to come up with an algorithm such that, given a binary tree and a sum, the algorithm returns true if the tree has a root-to-leaf path such that adding up all the values along the path equals the given sum. The algorithm returns false if no such path can be found.

The simplest approach seems to be the one where we traverse all the root-to-leaf paths, summing up the value along the path and checking when we reach a leaf, whether the sum matches the given value. A simple trick can reduce the processing time of the above algorithm. Instead of summing up the values from zero as we traverse each node along the root-to-leaf-path, we choose to subtract the value from the given sum value, as we traverse each node and pass the remaining sum value to the next node on the

path. If the remaining sum becomes zero or negative before we hit the leaf, we can terminate further traversal along that path and start exploring the next path. If the sum becomes zero, on traversing the leaf, we can return the answer 'true' since we know that the path-sum along the path matches the given sum value. Shown below is the pseudo code for this problem:

```

int hasPathSumEqualToX(int proposed_sum, struct node* tree)
{
    // return true if we have traversed past the leaf and the sum is equal to 0

    if (node == NULL)
    {
        return (sum == 0);
    }
    else
    {
        // otherwise check both subtrees
        int remaining_sum = sum - node->data;
        return (hasPathSum(remaining_sum, node->left) ||
                hasPathSum(remaining_sum, node->right));
    }
}

```

What is the complexity of the above algorithm? Is it possible to come up with a better algorithm? A variant of the path-sum problem is to enumerate all the paths from root-to-leaf in a binary tree. What is the worst-case time complexity of an algorithm that needs to enumerate all the root-to-leaf paths? I leave these questions for readers to think about.

This month's takeaway problem

This month, let us stay with the binary trees. A 'binary search tree' (BST) or 'ordered binary tree' is a type of binary tree where the nodes are arranged in order: for each node in the tree, all elements in the left sub-tree of the node are less-or-equal to the value of the node, and all the elements in the right sub-tree of the node are greater than the node. Binary search trees are used for element insertion and look-up.

This month's problem is to determine whether a given binary tree is, in fact, a binary search tree or not. Given a plain binary tree, you have to examine the tree to determine if it meets the requirements of a binary search tree. To be a binary search tree, at every node, all of the nodes in its left tree must be $\leq$ the node, and all of the nodes in its right subtree must be $>$ the node. Your algorithm should return true if the binary tree meets the criteria for a binary search tree. Else it should return false.

If you have any favourite programming puzzles that you would like to discuss on this forum, please send them to me. Feel free to send your solutions and feedback to sandyasm_AT_yahoo_DOT_com. Till we meet again next month, happy programming! 

About the author:

Sandya Mannarswamy. The author is a specialist in compiler optimisation and works at Hewlett-Packard India. She has a number of publications and patents to her credit, and her areas of interest include virtualisation technologies and software development tools.